

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch: B.Tech Artificial Intelligence & Data Science Semester: V

Subject Code & Name: CS3807 – Deep Learning Laboratory AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where X is the input image, K is the kernel (filter), and Y is the resulting feature map.

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where N is the input size, F is the filter size, P is the padding, and S is the stride.

2.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

Table 1: Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

3 Numerical Examples

3.1 Numerical Example 1: Convolution

Input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Sliding the kernel over the input gives

$$1(1) + 2(0) + 4(0) + 5(1) = 6, \quad 2(1) + 3(0) + 5(0) + 6(1) = 8,$$

$$4(1) + 5(0) + 7(0) + 8(1) = 12, \quad 5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

3.2 Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Applying a 2×2 max pooling filter over the four non-overlapping windows gives 8, 3, 6, and 9 respectively, producing the output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

3.3 Numerical Example 3: Parameter Calculation

For an input image of size $32 \times 32 \times 3$ passed through a convolution layer with 16 filters of size 3×3 , the number of trainable parameters is

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

4 Dataset

The CIFAR-10 dataset was used for this experiment. It was loaded using the TensorFlow/Keras dataset API and consists of 50,000 training images and 10,000 testing images, each of size $32 \times 32 \times 3$ (RGB), distributed uniformly across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck (5,000 training images per class). The pixel values of all images were normalized to the range $[0, 1]$ by dividing by 255, and the integer class labels were one-hot encoded using `tf.keras.utils.to_categorical` for use with the categorical cross-entropy loss.

5 Experimental Procedure

- Loaded CIFAR-10 via kagglehub module
- Displayed ten sample images with their labels and plotted the class distribution verified the dataset is perfectly balanced across all ten classes.
- Applied a single convolution layer to a sample image with three kernel sizes (3×3 , 5×5 , 7×7) to see how the feature map shrinks as the kernel grows.
- Repeated the same idea for stride and padding: varied stride (1, 2) and padding (`same`, `valid`) on a 3×3 kernel to compare output sizes.
- After training the main network, visualized the feature maps from its first convolution layer to see what the filters had picked up on.
- Trained two identical CNNs – one with max pooling, one with average pooling – and compared test accuracy, loss, and training time.
- Built and trained the main CNN:

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

using the Adam optimizer, 20 epochs, batch size 32.

- Evaluated the trained model on the test set: accuracy, precision, recall, F1-score, classification report, and confusion matrix.

6 Model Architecture

- **Input:** $32 \times 32 \times 3$
- **Conv2D:** 32 filters, 3×3 , ReLU, `same` padding
- **MaxPooling2D:** 2×2
- **Conv2D:** 64 filters, 3×3 , ReLU, `same` padding
- **MaxPooling2D:** 2×2
- **Flatten**
- **Dense:** 128 units, ReLU
- **Dense (output):** 10 units, Softmax
- **Optimizer:** Adam **Loss:** categorical cross-entropy
- **Epochs:** 20 **Batch size:** 32
- **Trainable parameters:** 545,098 **Training time:** ≈ 144.8 s

7 Experimental Results

- Final training accuracy: **94.32%**
- Final validation/testing accuracy: **64.59%**
- Testing loss: **2.684**
- Precision (macro avg): **0.664** Recall (macro avg): **0.646** F1-score (macro avg): **0.648**
- Trainable parameters: **545,098**
- Full per-class breakdown in Table 2; confusion matrix in Figure 8

Table 2: Classification Report

Class	Precision	Recall	F1-score	Support
airplane	0.67	0.74	0.70	1000
automobile	0.83	0.71	0.76	1000
bird	0.54	0.61	0.57	1000
cat	0.44	0.47	0.45	1000
deer	0.59	0.57	0.58	1000
dog	0.55	0.57	0.56	1000
frog	0.83	0.56	0.66	1000
horse	0.79	0.67	0.72	1000
ship	0.60	0.88	0.72	1000
truck	0.80	0.69	0.74	1000
Accuracy		0.65		10000
Macro avg	0.66	0.65	0.65	10000
Weighted avg	0.66	0.65	0.65	10000

8 Plots

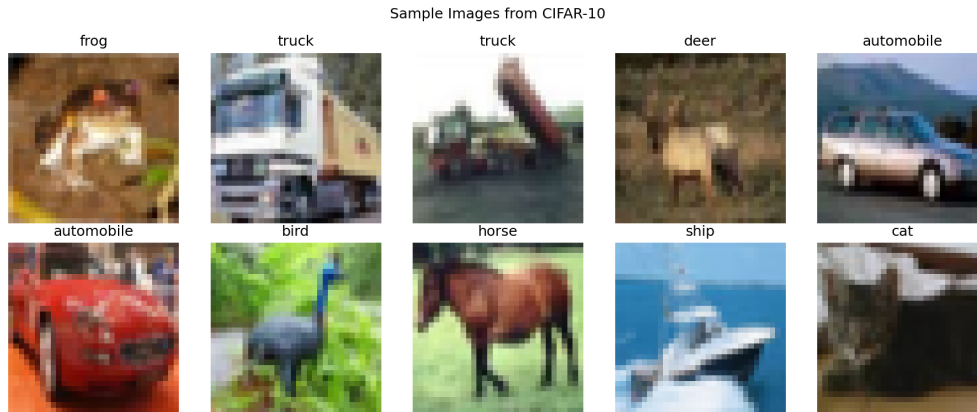


Figure 1: Ten sample images from the CIFAR-10 training set with their class labels.

- CIFAR-10 images are low-resolution (32×32), covering a mix of vehicles and animals.
- Objects stay visually distinguishable despite the small size, supporting CNN-based classification.

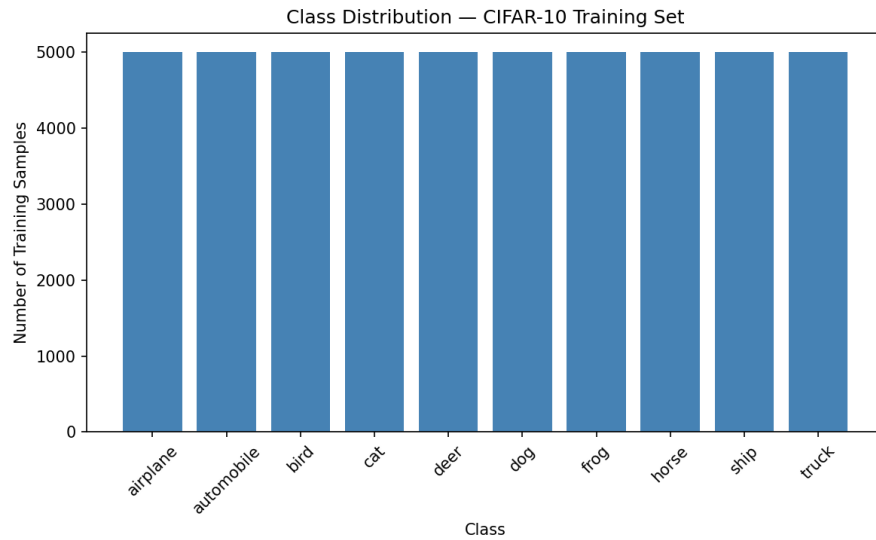


Figure 2: Class distribution of the CIFAR-10 training set.

- All ten classes contain exactly 5,000 training images each – the dataset is perfectly balanced.
- Accuracy is therefore a fair overall metric, and no class re-weighting or resampling was needed.

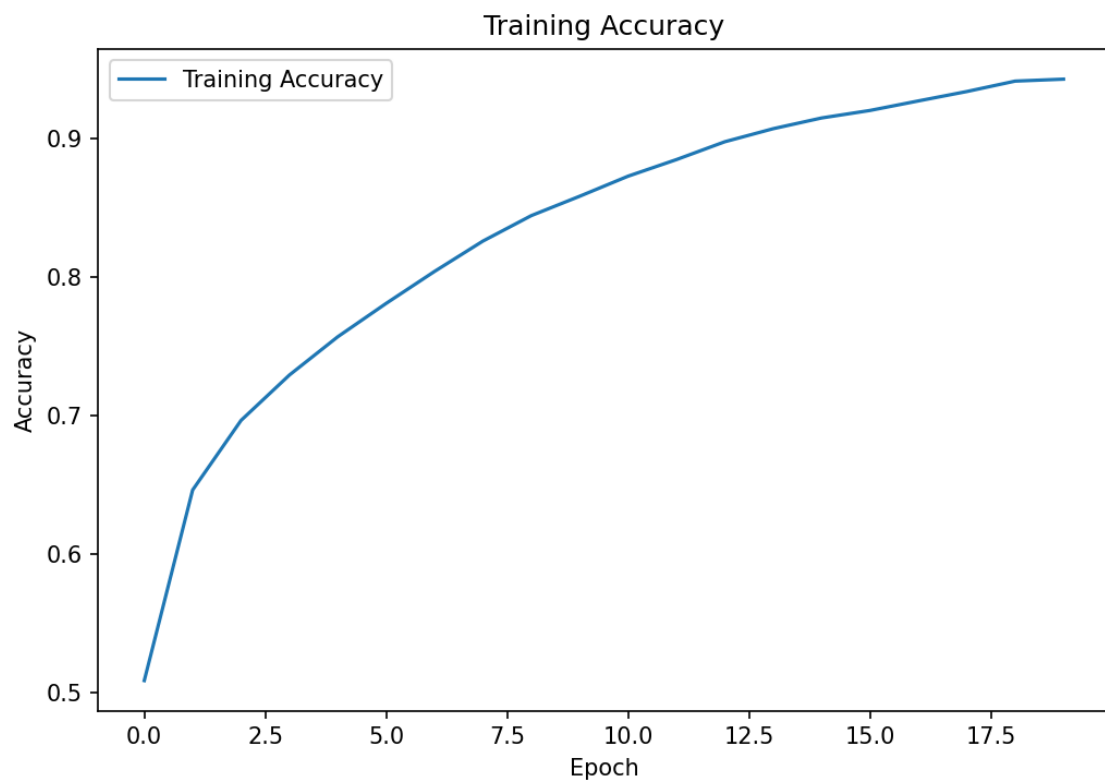


Figure 3: Training accuracy over 20 epochs.

- Rose steadily from about 51% (epoch 1) to 94.3% (epoch 20).
- The smooth, uninterrupted climb shows stable optimization with no signs of instability.

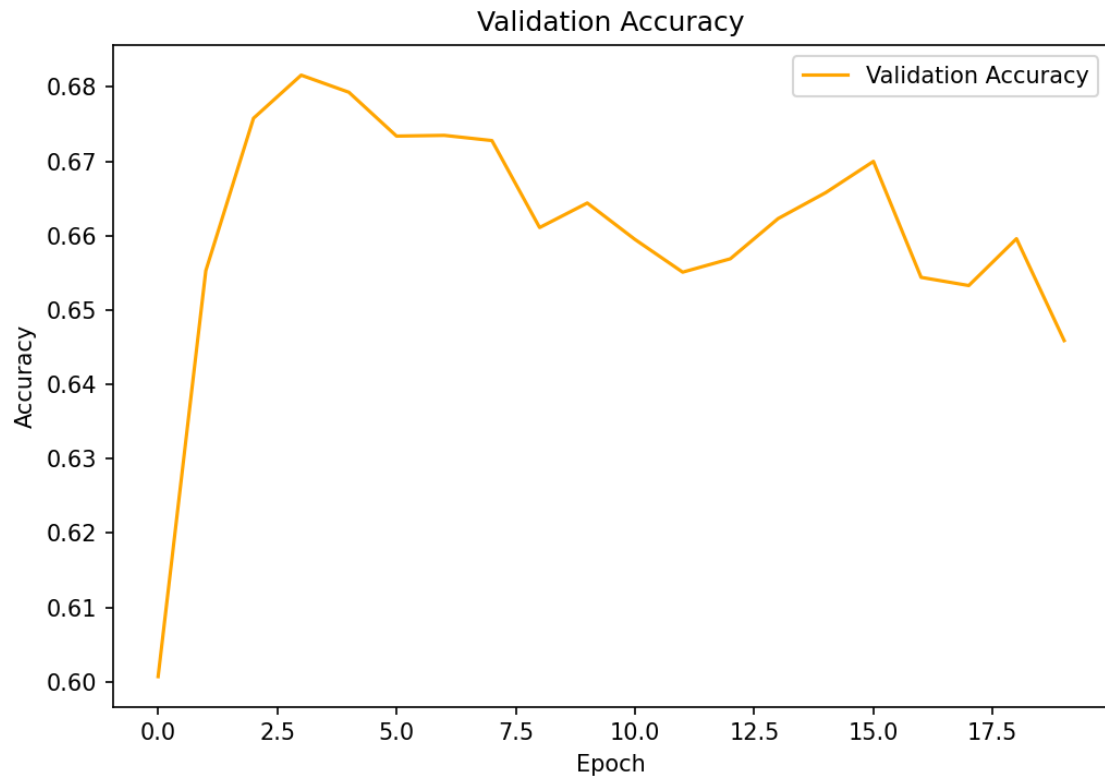


Figure 4: Validation accuracy over 20 epochs.

- Rose sharply for the first 3 epochs, peaking near 68.2%.
- Plateaued, then drifted down to 64.6% by epoch 20.
- This divergence from the still-climbing training accuracy signals overfitting starting around epoch 3–4.

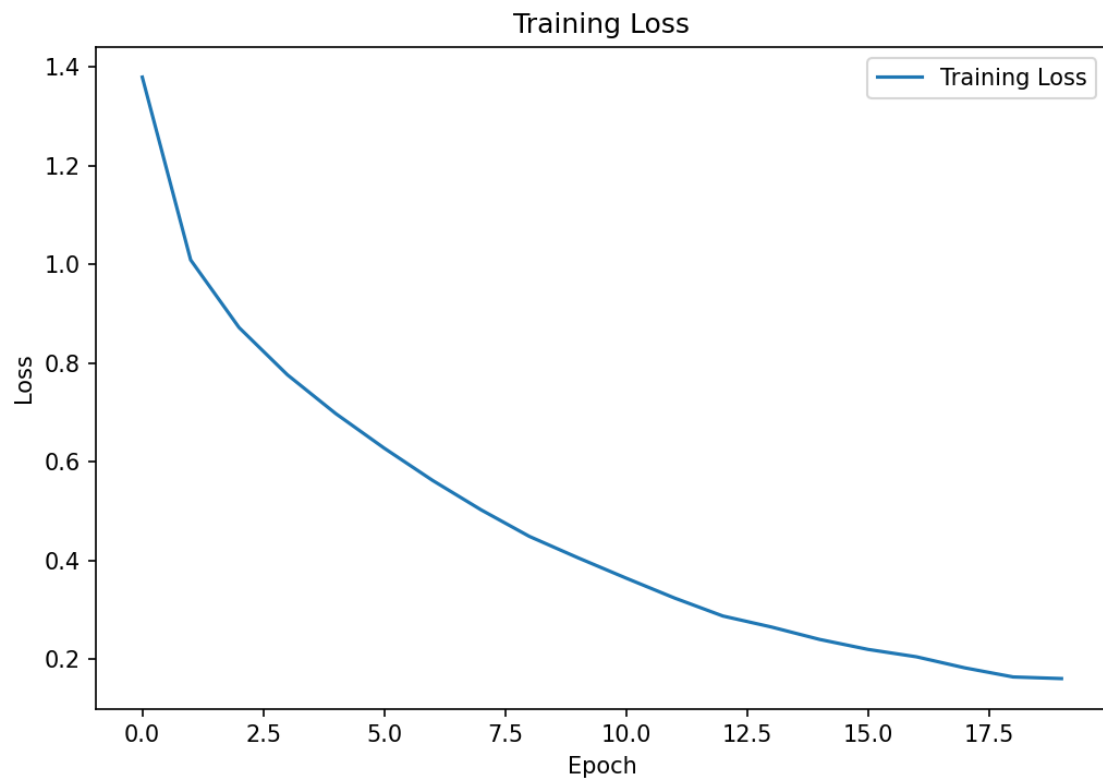


Figure 5: Training loss over 20 epochs.

- Fell consistently across all 20 epochs, from about 1.38 to 0.16.
- Shows the network kept minimizing training error for the full run.

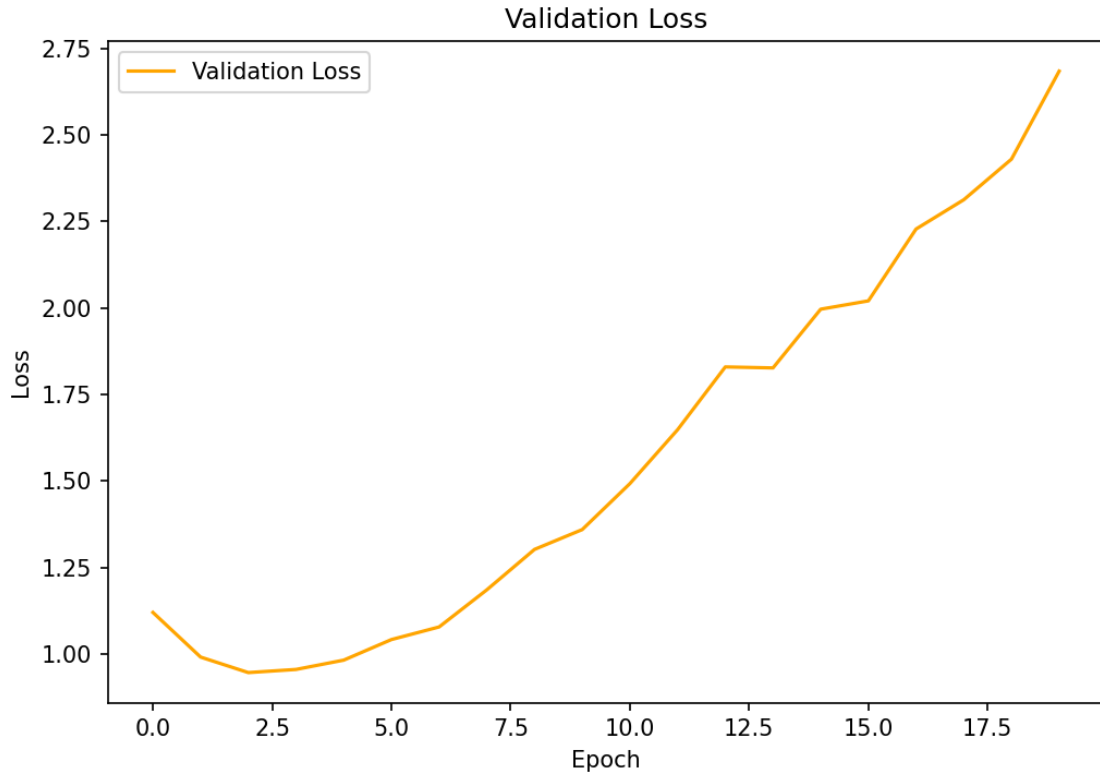


Figure 6: Validation loss over 20 epochs.

- Hit its minimum (≈ 0.95) around epoch 2–3.
- Rose steadily afterward, reaching 2.68 by epoch 20.
- Confirms overfitting after the first few epochs, alongside the plateauing validation accuracy.

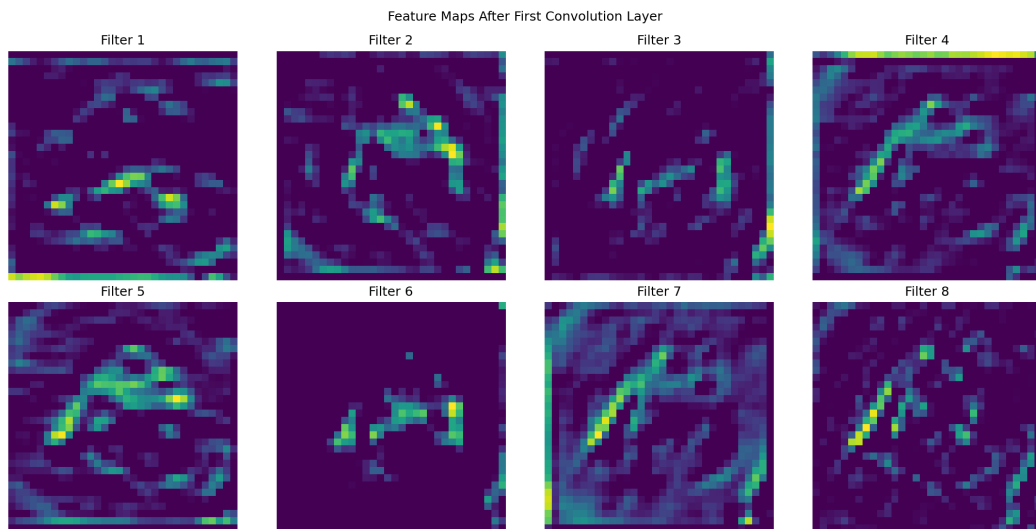


Figure 7: Feature maps produced by the first convolution layer for a sample input image.

- Different filters respond to different spatial structures – mostly edges and outlines against the background.
- Filter 6 activates sparsely on one small region; Filter 7 responds strongly along image borders.
- Overall, the filters have specialized to pick up different low-level visual patterns.

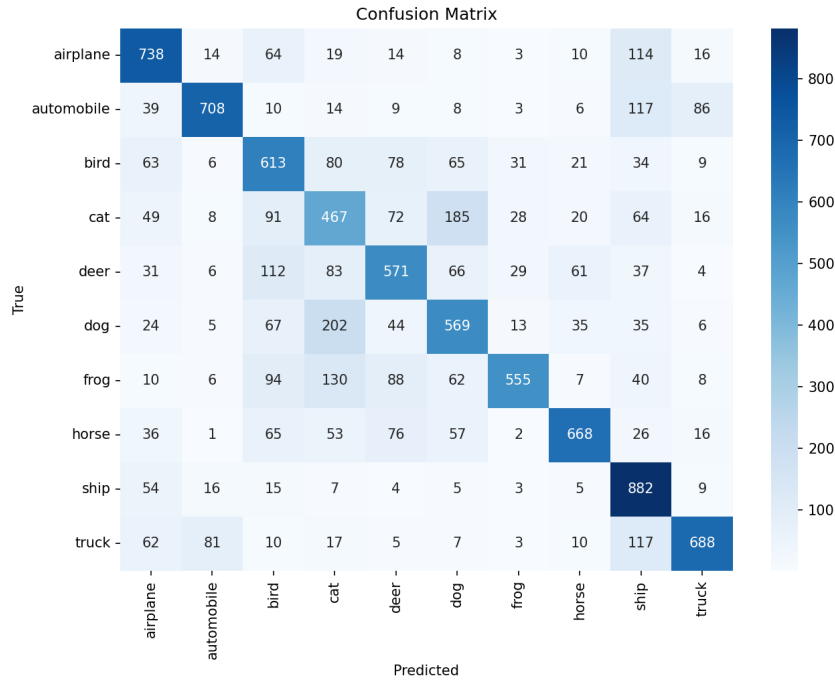


Figure 8: Confusion matrix on the CIFAR-10 test set.

- Strongest on visually distinctive classes: ship (882 correct), airplane (738 correct).
- Weakest on cat, often confused with dog (185 misclassifications).
- Vehicle classes (airplane, automobile, ship, truck) get mixed up with each other, as do the animal classes (bird, cat, deer, dog), which share similar textures and poses.

9 Kernel Comparison

- A convolution layer (8 filters, **valid** padding) was applied to a sample image using 3×3 , 5×5 , and 7×7 kernels.
- Feature map size shrinks as the kernel grows: $30 \times 30 \rightarrow 28 \times 28 \rightarrow 26 \times 26$.
- Parameter count rises with kernel size too: $224 \rightarrow 608 \rightarrow 1,184$.

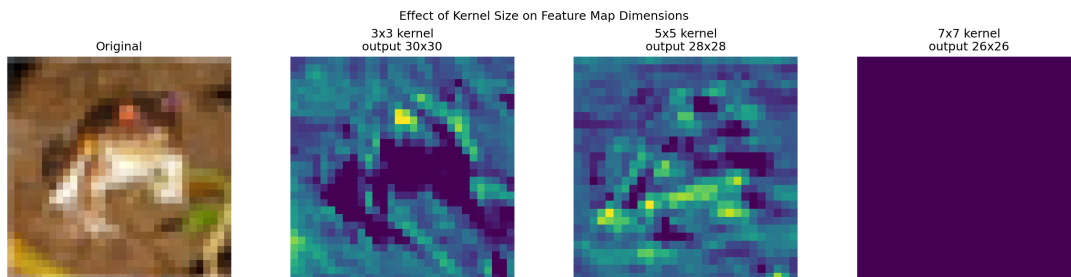


Figure 9: Effect of kernel size (3×3 , 5×5 , 7×7) on feature map dimensions.

- Larger kernels see a wider receptive field and capture more global structure per layer, but shrink the output more and cost more parameters.
- Smaller kernels (3×3) keep more spatial resolution and are cheaper – hence their use in the main CNN.

Table 3: Kernel Size Comparison

Kernel Size	Output Feature Map Shape	Parameters (8 filters)
3×3	$30 \times 30 \times 8$	224
5×5	$28 \times 28 \times 8$	608
7×7	$26 \times 26 \times 8$	1,184

10 Hyperparameter Analysis

- Studied the effect of stride and padding using the same 3×3 convolution layer on the sample image (Table 4).

Table 4: Effect of Stride and Padding on Output Dimensions

Stride	Padding	Output Shape
1	valid	$30 \times 30 \times 8$
2	valid	$15 \times 15 \times 8$
1	same	$32 \times 32 \times 8$
2	same	$16 \times 16 \times 8$

- Doubling stride ($1 \rightarrow 2$) roughly halves the output size, since the kernel skips more pixels each step – lowers both resolution and computation.
- same** padding keeps the output size equal to the input at stride 1; **valid** padding has no padding and always shrinks the output.
- That’s why **same** padding was used in the main CNN – it lets the network go deeper without the feature maps vanishing.

11 Pooling Comparison

- Two identical CNNs – one max pooling, one average pooling – trained under the same conditions (Adam, 20 epochs, batch size 32); results in Table 5.

Table 5: Max Pooling vs. Average Pooling

Pooling Type	Test Accuracy	Test Loss	Training Time (s)
Max Pooling	64.59%	2.684	144.8
Average Pooling	65.64%	2.183	130.7

- Both use a 2×2 window, so output size and parameter count (545,098) are identical – pooling layers have no trainable weights of their own.
- Average pooling edged out max pooling: 65.64% vs. 64.59% test accuracy, and a lower test loss (2.183 vs. 2.684).
- Average pooling also trained marginally faster.
- Suggests smoother, averaged activations generalized slightly better here than max pooling which may have overfit more.

12 Results Table

Table 6: Summary of Final Experimental Results

Metric	Value
Training Accuracy	94.32%
Testing Accuracy	64.59%
Precision (macro avg)	0.664
Recall (macro avg)	0.646
F1-score (macro avg)	0.648
Number of Trainable Parameters	545,098

13 Additional Exercises

1. For a 64×64 input image with a 5×5 kernel, stride 2, and padding 2, the output size is $\frac{64-5+2(2)}{2} + 1 = 32$, giving a 32×32 output.
2. For a convolution layer with 64 filters of size 3×3 applied to an RGB input, the number of trainable parameters is $(3 \times 3 \times 3 + 1) \times 64 = 1,792$.
3. ReLU converges faster and avoids the vanishing gradient problem that affects Sigmoid for deep networks, since its gradient is either 0 or 1 rather than shrinking toward zero for large inputs. Sigmoid saturates at both extremes, slowing learning, whereas ReLU's simplicity also makes it computationally cheaper.
4. Replacing max pooling with average pooling (Section 7) slightly improved test accuracy (65.64% vs. 64.59%) and reduced test loss, while training marginally faster, indicating average pooling was mildly beneficial for this architecture and dataset.
5. Increasing the number of convolution filters generally increases the model's representational capacity and its trainable parameter count (and therefore training time), which can improve accuracy up to a point but also raises the risk of overfitting on a fixed-size dataset such as CIFAR-10, consistent with the overfitting already observed with the smaller filter counts used in this experiment.

14 Discussion

1. **Why is convolution preferred over fully connected layers for images?** Convolution exploits local spatial correlation and parameter sharing across the image, drastically reducing the number of parameters compared to a fully connected layer while preserving spatial structure.
2. **How does stride affect the feature map size?** A larger stride causes the kernel to move across the input in bigger steps, reducing the output feature map's spatial dimensions and the amount of overlap between adjacent receptive fields.
3. **What is the role of padding?** Padding adds extra pixels around the input's border so that the output feature map can retain the input's spatial dimensions and so that border pixels are not under-represented in the convolution.
4. **Why is pooling used?** Pooling reduces the spatial dimensions of feature maps, which lowers computational cost and the number of parameters in subsequent layers.
5. **How do feature maps represent image characteristics?** Each feature map is the convolution of a learned filter to the input, highlighting spatial regions where that filter's pattern (such as an edge or texture) is present.
6. **Why do CNNs require fewer parameters than MLPs?** CNNs share the same small set of kernel weights across every spatial location in the image, whereas an MLP would require a separate weight for every pixel-to-neuron connection and CNN use pooling layers to reduce the dimensions and number of parameters.

15 Conclusion

- Built and evaluated a CNN on CIFAR-10, covering convolution, pooling, and feature map visualization.
- Final testing accuracy: 64.59%, macro F1-score: 0.648 after 20 epochs.
- Training accuracy (94.32%) far outpaced testing accuracy, and validation loss rose sharply after epoch 3 – clear signs of overfitting.

16 Additional Experiment: CNN Trained for 3 Epochs

The same CNN architecture and preprocessing steps were used, but the model was trained for only 3 epochs. The objective was to compare its performance with the previous experiments while further reducing the training time.

Table 7: Results of the 3-Epoch CNN

Metric	Value
Training Accuracy	69.19%
Testing Accuracy	67.45%
Training Loss	0.886
Validation Loss	0.939
Precision (Macro Avg.)	0.676
Recall (Macro Avg.)	0.675
F1-score (Macro Avg.)	0.667
Trainable Parameters	545,098
Training Time	27.77 s

The model achieved a testing accuracy of 67.45% while completing training in about 28 seconds. The training and validation losses were close, indicating better generalization than the longer training runs.

16.1 Pooling Comparison

Table 8: Max Pooling vs. Average Pooling (3 Epochs)

Pooling Type	Test Accuracy	Test Loss	Training Time (s)
Max Pooling	67.45%	0.939	27.77
Average Pooling	65.91%	0.975	25.71

Max pooling achieved slightly better accuracy and a lower test loss than average pooling in this experiment.

16.2 Comparison with the 20-Epoch Model

- The 3-epoch model achieved a higher testing accuracy (67.45%) than the 20-epoch model (64.59%).
- Training time was reduced from about 145 seconds to only 28 seconds.
- Compared to the 20-epoch model, the 3-epoch model showed better generalization, with training and validation losses remaining much closer.

17 References

1. Goodfellow, I., Bengio, Y., & Courville, A., *Deep Learning*.
2. Bishop, C. M., *Pattern Recognition and Machine Learning*.
3. Haykin, S., *Neural Networks and Learning Machines*.
4. TensorFlow Documentation, <https://www.tensorflow.org>.
5. CIFAR-10 Dataset Documentation, <https://www.cs.toronto.edu/~kriz/cifar.html>.